## WEEK 8: Banker's Algorithm for Deadlock Avoidance

```
#include <stdio.h>

int main()
{
   int n, m;
   int allocation[10][10], max[10][10];
   int need[10][10], available[10];
   int work[10], finish[10] = {0};
   int safeSequence[10];
   int count = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   printf("\nEnter Allocation Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++)
         scanf("%d", &allocation[i][j]);

   printf("\nEnter Maximum Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++)
         scanf("%d", &max[i][j]);

   printf("\nEnter Available Resources:\n");
   for (int j = 0; j < m; j++)
      scanf("%d", &available[j]);

   /* Calculate Need Matrix */
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++)
         need[i][j] = max[i][j] - allocation[i][j];

   /* Initialize Work */
   for (int j = 0; j < m; j++)
      work[j] = available[j];

   /* Safety Algorithm */
   while (count < n)
   {
      int found = 0;
```

```
for (int i = 0; i < n; i++)
{
   if (finish[i] == 0)
   {
      int possible = 1;

      for (int j = 0; j < m; j++)
      {
         if (need[i][j] > work[j])
         {
            possible = 0;
            break;
         }
      }

      if (possible)
      {
         for (int j = 0; j < m; j++)
            work[j] += allocation[i][j];

         safeSequence[count++] = i;
         finish[i] = 1;
         found = 1;
      }
   }
}
```

Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE STATE.

## WEEK9: C Program: Simulate Banker's Algorithm for Deadlock Prevention

```
#include <stdio.h>

int main()
{
    int n, m;
    int allocation[10][10];
    int max[10][10];
    int need[10][10];
    int available[10];
    int work[10];
    int finish[10] = {0};
    int safeSequence[10];

    int i, j, count = 0;
    int found, canExecute;

    // Read number of processes
    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Read number of resources
    printf("Enter number of resources: ");
    scanf("%d", &m);

    // Read Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
```

### Output

```
Need Matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

System is in SAFE STATE.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

## WEEK 10:

## C Programs for File Allocation Strategies

The three common file allocation methods are:

1. **Sequential (Contiguous) Allocation**
2. **Linked Allocation**
3. **Indexed Allocation**

Below are simple C programs suitable for an **Operating Systems lab**.

---

### 1. Sequential File Allocation

In sequential allocation, a file occupies **contiguous blocks** on the disk.

```
#include <stdio.h>

int main()
{
    int disk[50] = {0};
    int start, length, i;

    printf("Enter starting block: ");
    scanf("%d", &start);

    printf("Enter number of blocks required: ");
    scanf("%d", &length);

    /* Check whether blocks are available */
    for (i = start; i < start + length; i++)
    {
        if (disk[i] == 1)
```

```
            {
                printf("File cannot be allocated.\n");
                return 0;
            }
        }

        /* Allocate blocks */
        for (i = start; i < start + length; i++)
        {
            disk[i] = 1;
        }

        printf("\nFile allocated successfully.\n");
        printf("Allocated Blocks: ");

        for (i = start; i < start + length; i++)
        {
            printf("%d ", i);
        }

        printf("\n");

        return 0;
}
```

**Output**

```
Enter starting block: 10
Enter number of blocks required: 5

File allocated successfully.
Allocated Blocks: 10 11 12 13 14
```

---

## 2. Linked File Allocation

In linked allocation, file blocks can be located anywhere on the disk. Each block contains a **pointer to the next block**.

```
#include <stdio.h>

int main()
{
    int n, i;
    int blocks[50];

    printf("Enter number of blocks required: ");
```

```
    scanf("%d", &n);

    printf("Enter the block numbers:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &blocks[i]);
    }

    printf("\nFile allocated successfully.\n");

    printf("Linked Allocation:\n");

    for (i = 0; i < n; i++)
    {
        printf("%d", blocks[i]);

        if (i < n - 1)
            printf(" -> ");
    }

    printf(" -> NULL\n");

    return 0;
}
```

**Output**

```
Enter number of blocks required: 5

Enter the block numbers:
7
19
2
25
11

File allocated successfully.

Linked Allocation:
7 -> 19 -> 2 -> 25 -> 11 -> NULL
```

---

### 3. Indexed File Allocation

In indexed allocation, an **index block** contains the addresses of all blocks belonging to a file.

```
#include <stdio.h>

int main()
{
    int indexBlock;
    int n, i;
    int blocks[50];

    printf("Enter index block number: ");
    scanf("%d", &indexBlock);

    printf("Enter number of blocks required: ");
    scanf("%d", &n);

    printf("Enter the block numbers:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &blocks[i]);
    }

    printf("\nFile allocated successfully.\n");

    printf("Index Block: %d\n", indexBlock);

    printf("Blocks allocated through index block:\n");

    for (i = 0; i < n; i++)
    {
        printf("%d ", blocks[i]);
    }

    printf("\n");

    return 0;
}
```

**Output**

```
Enter index block number: 5
Enter number of blocks required: 4
```